

Data Collection & Preprocessing

(Natural Language Processing)

Randil Pushpananda, PhD





Data collection facts

- Data collection, cleaning and preparation are estimated to take 70% of any data science task
- Quality and diversity of data play a significant role
- Sources of data are disparate (very different one from another)
- Defining clear objectives is essential for guiding the data collection process effectively



Potential Data Sources

- Data sources impact the quality, diversity, and applicability of the collected data
- Identify the kind of data you want to collect?
 - Text?
 - Numerical?
 - Speech?



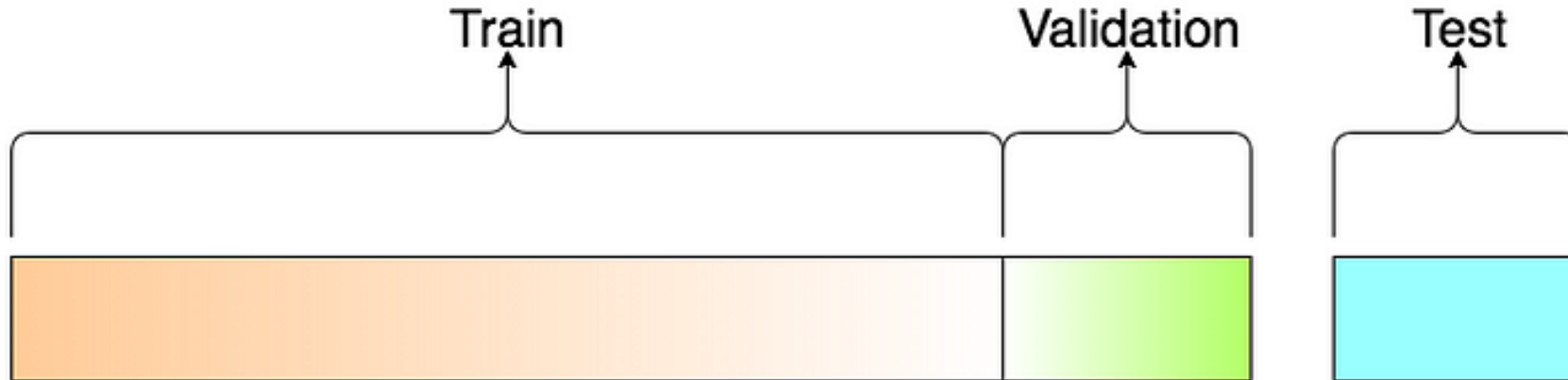
First Things First: Look at Your Data

REMEMBER

- We cannot use the data we used to build the model to evaluate it since the developed model always remembers the whole training dataset.
- Target is to generalize the model (Check whether the model performs well on new data)
- Split Dataset:
 - Training
 - Validation
 - Testing

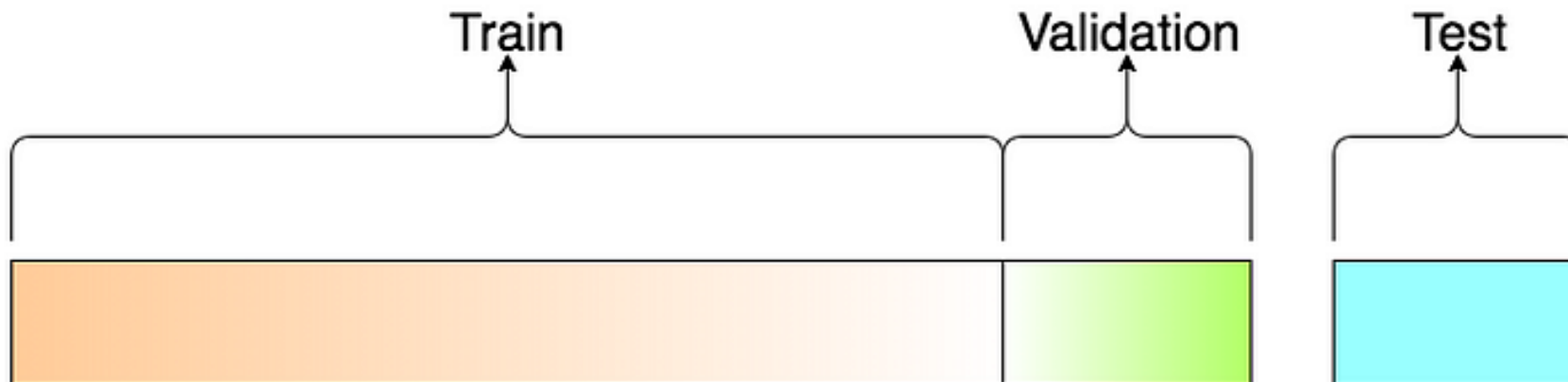
Training Dataset

- One part of the data is used to build our machine learning model
- The model *sees* and *learns* from this data.



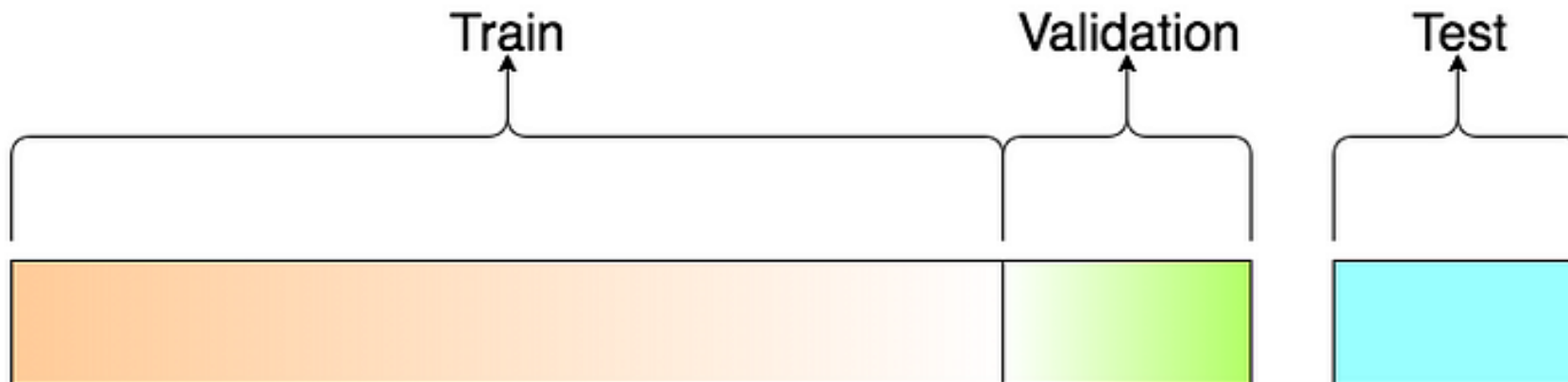
Validation Dataset

- The sample of data used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters.
- Use this data to fine-tune the model hyperparameters.
- The model occasionally sees this data, but never does it “Learn” from this.



Test Dataset

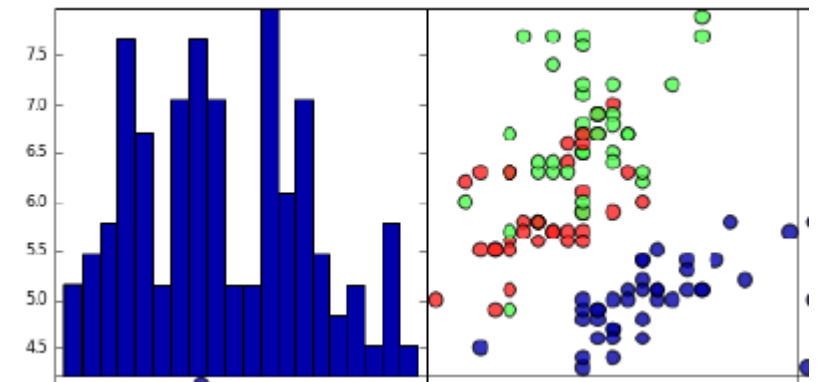
- Used to assess how well the model works.
- Used to provide an unbiased evaluation of a final model fit on the training dataset
- Only used once a model is completely trained(using the train and validation sets)



First Things First: Look at Your Data

REMEMBER:

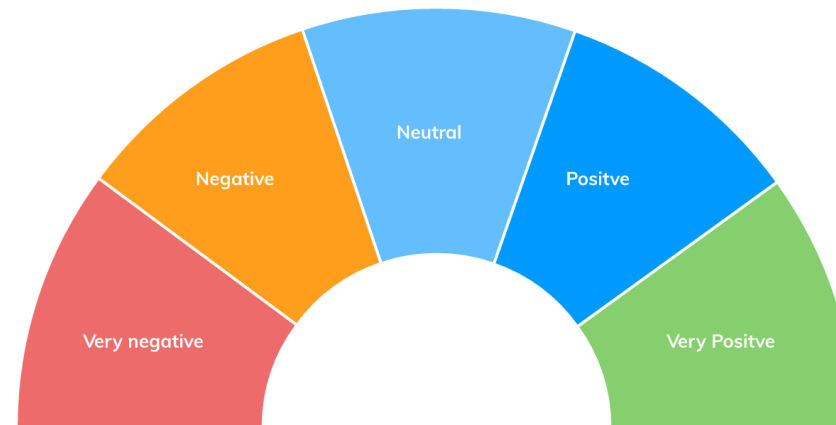
- Before building a machine learning model it is often a good idea to inspect the data, to see **if the task is easily solvable without machine learning**, or **if the desired information might not be contained in the data**.
- One of the best ways to inspect data is to visualize it.
 - Scatter Plots
 - Box and Whisker Plot for Large Data
 - Pie and Donut Charts



First Things First: Look at Your Data

Text Visualization/Analysis:

- Word Clouds
- Topic classification
- Sentiment Analysis/Aspect Based Sentiment Analysis



What kind of Data?

How much Data?

How to collect a Balanced Dataset?

Study the Dataset and Identify Features

How to divide Training, Validation and
Testing Data?

What are the preprocessing steps required?

Features of a Dataset

- garbage in → garbage out
- System will only be capable of learning if the training data contains enough relevant features and not too many irrelevant ones.
- A critical part of the success of a machine learning project is coming up with a good set of features to train on. This process, called *feature engineering*
 - Feature selection
 - Feature extraction
 - Creating new features by gathering new data

Potential Data Sources

- Identify reliable data sources for the data you want to collect
 - Look for industry-accepted or professional data sources in your domain
 - Look for highly cited sources
 - Make a list of all possible sources & weigh the pros and cons
 - Services offered by data providers
 - Payment plan, if any
 - Frequency of update
 - Attributes of dataset
- Identify how the data might be collected legally from these sources



Potential Data Sources

Websites and Online Platforms:

- Websites, blogs, news articles, social media platforms, discussion forums, online communities.
- Example: Gathering tweets for sentiment analysis or news articles for summarization.

Text Corpora and Datasets:

- Curated datasets created for specific tasks, such as sentiment analysis, machine translation, question answering.
- Example: IMDb dataset for sentiment analysis, WMT translation datasets.

Potential Data Sources

Domain-Specific Data:

- Data from a particular domain, industry, or field (e.g., medical journals, legal documents).
- Example: Medical research articles for medical NLP tasks.

User-Generated Content:

- Content generated by users, such as customer reviews, product feedback, and online comments.
- Example: Amazon product reviews for sentiment analysis or user opinions.

Scientific Journals and Publications:

- Academic and research papers, publications in specific domains.
- Example: Academic papers for building domain-specific language models.

Potential Data Sources

- **Factors to Consider:**
 - **Relevance:** Ensure the data source aligns with the project's objectives and intended tasks.
 - **Quality:** Choose sources with credible, accurate, and well-written content.
 - **Quantity:** Know the amount of data needed.
 - **Format:** Data should be in a format easy to work with.
 - **Diversity:** Opt for sources that provide a diverse range of language styles, topics, and perspectives.
 - **Accessibility:** Consider data accessibility, licensing, and terms of use.



Data Collection Methods

Using APIs:

- Utilizing APIs provided by various platforms to fetch structured data.
- **Sources:** Twitter API, Reddit API, News APIs (like NewsAPI), Google Books API.
- **Applications:** Collecting social media posts, news articles, and other structured text data.

Scraping the Web:

- Extracting text data from websites using automated scripts.
- **Tools:** BeautifulSoup, Scrapy, Selenium.
- **Applications:** Building large corpora from news websites, forums, blogs, and social media.

Data Collection Methods

Public Datasets:

- Using datasets that have been pre-collected and made available by researchers or organizations.
- **Examples:**
 - Text:** The Penn Treebank, WikiText, Project Gutenberg.
 - Speech:** LibriSpeech, Common Voice.
- **Applications:** Training models on well-established benchmarks.

Crowdsourcing:

- Gathering data by engaging human participants through platforms like Amazon Mechanical Turk.
- **Tools:** Amazon Mechanical Turk, Prolific, Figure Eight (formerly CrowdFlower).
- **Applications:** Annotating text for sentiment analysis, collecting paraphrases, creating question-answer pairs.

Data Collection Methods

Manual Data Entry:

- Directly inputting data by researchers or hired annotators.
- **Applications:** Creating gold-standard datasets, fine-tuning specific tasks where precision is crucial, speech transcription.

Sensor Data and IoT:

- Collecting speech data through devices like smart speakers.
- **Tools:** Home assistants (Amazon Echo, Google Home).
- **Applications:** Building voice recognition systems, contextual NLP tasks in smart environments.

Data Collection Methods

Document Scanning and OCR:

- Digitizing printed text using Optical Character Recognition (OCR).
- **Tools:** Tesseract, Adobe Acrobat OCR.
- **Applications:** Creating digital versions of books, historical documents, legal papers.

Surveys and Interviews:

- Collecting structured and unstructured text through direct interactions.
- **Applications:** Gathering user opinions, conducting sentiment analysis on feedback.

Speech Data Collection:

Call Centers and Customer Service Interactions

- Recording conversations between customers and service agents.
- Analyzing real-world dialogue, improving customer service automation, sentiment analysis.

In-Lab Recording

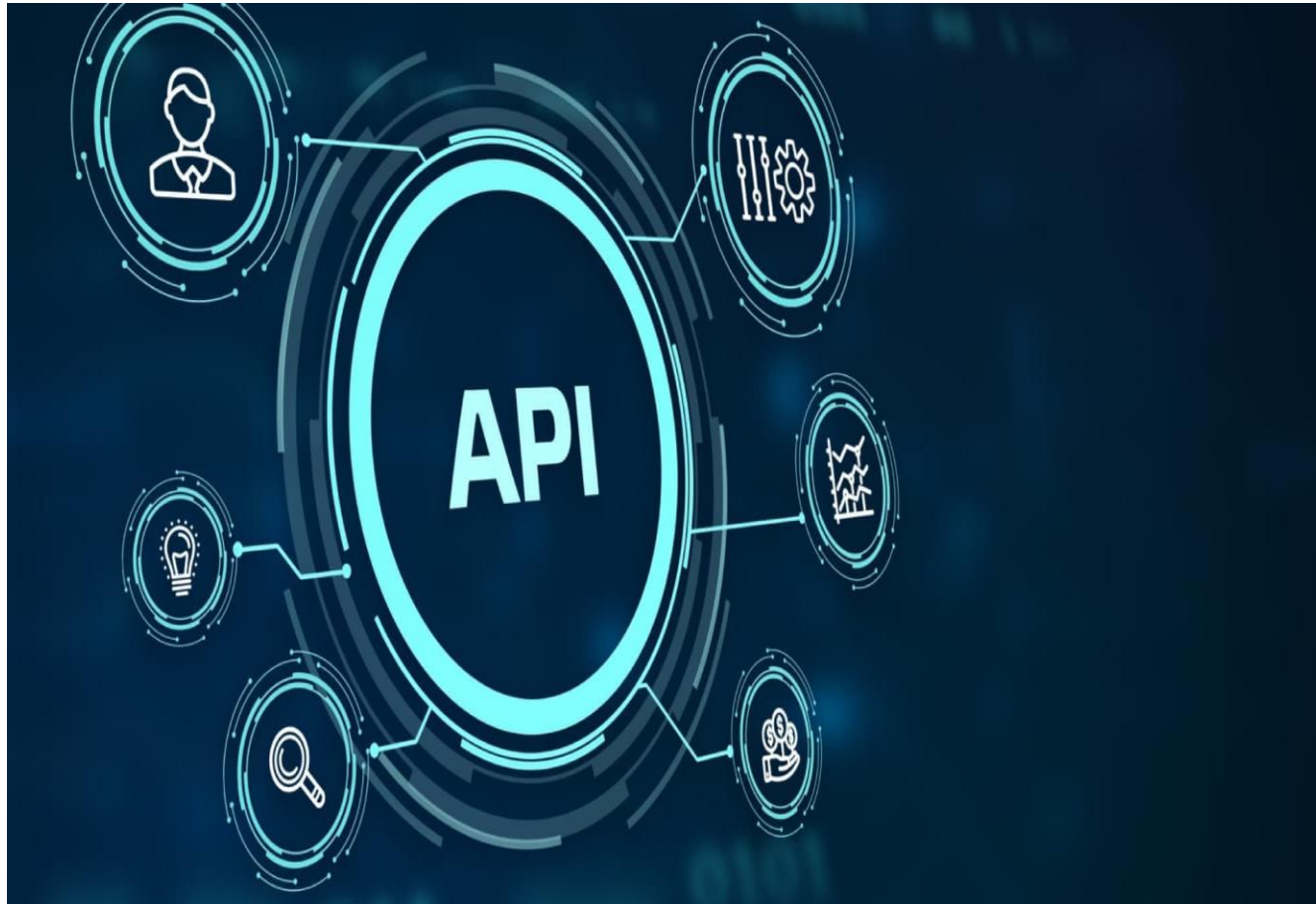
- Controlled environment recordings using high-quality equipment.
- Creating high-quality datasets, minimizing background noise.

Interview and Survey Recordings

- Recording responses from structured interviews or surveys.
- Collecting specific types of speech data, such as opinions, narratives, and structured responses.

Broadcast Media

- Extracting speech from TV shows, radio, podcasts, and YouTube videos.
- Building large corpora, analyzing different speaking styles and contexts.



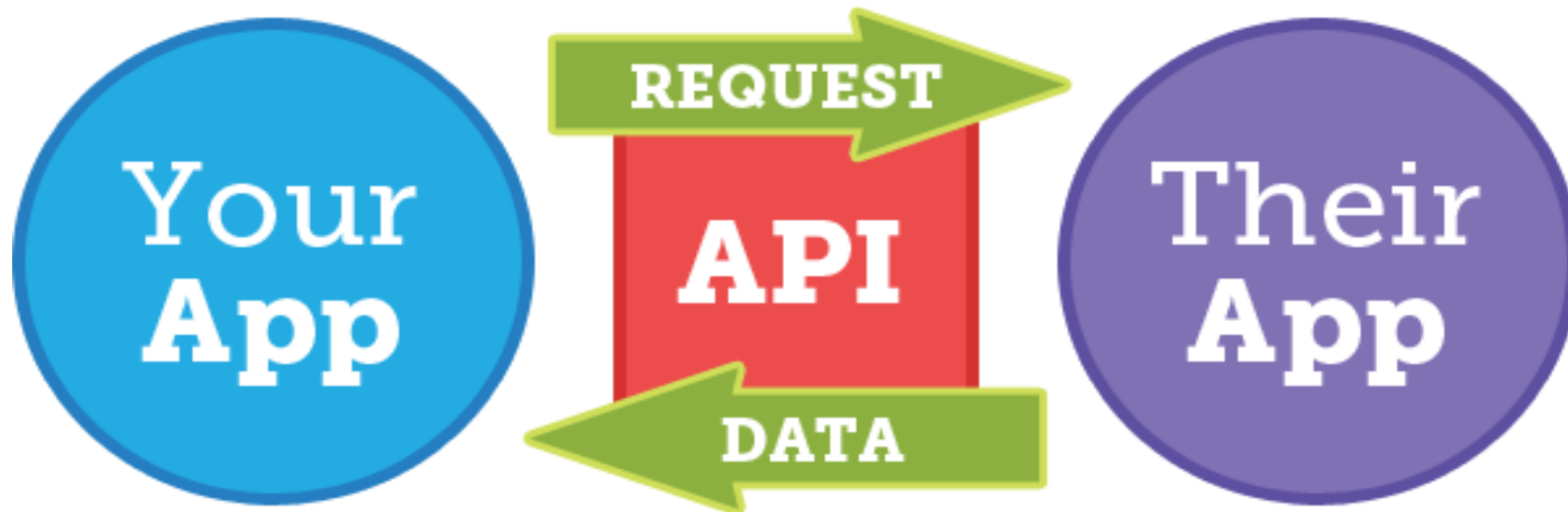
Data Collection through APIs

THE INTERNET IN 2023 EVERY MINUTE



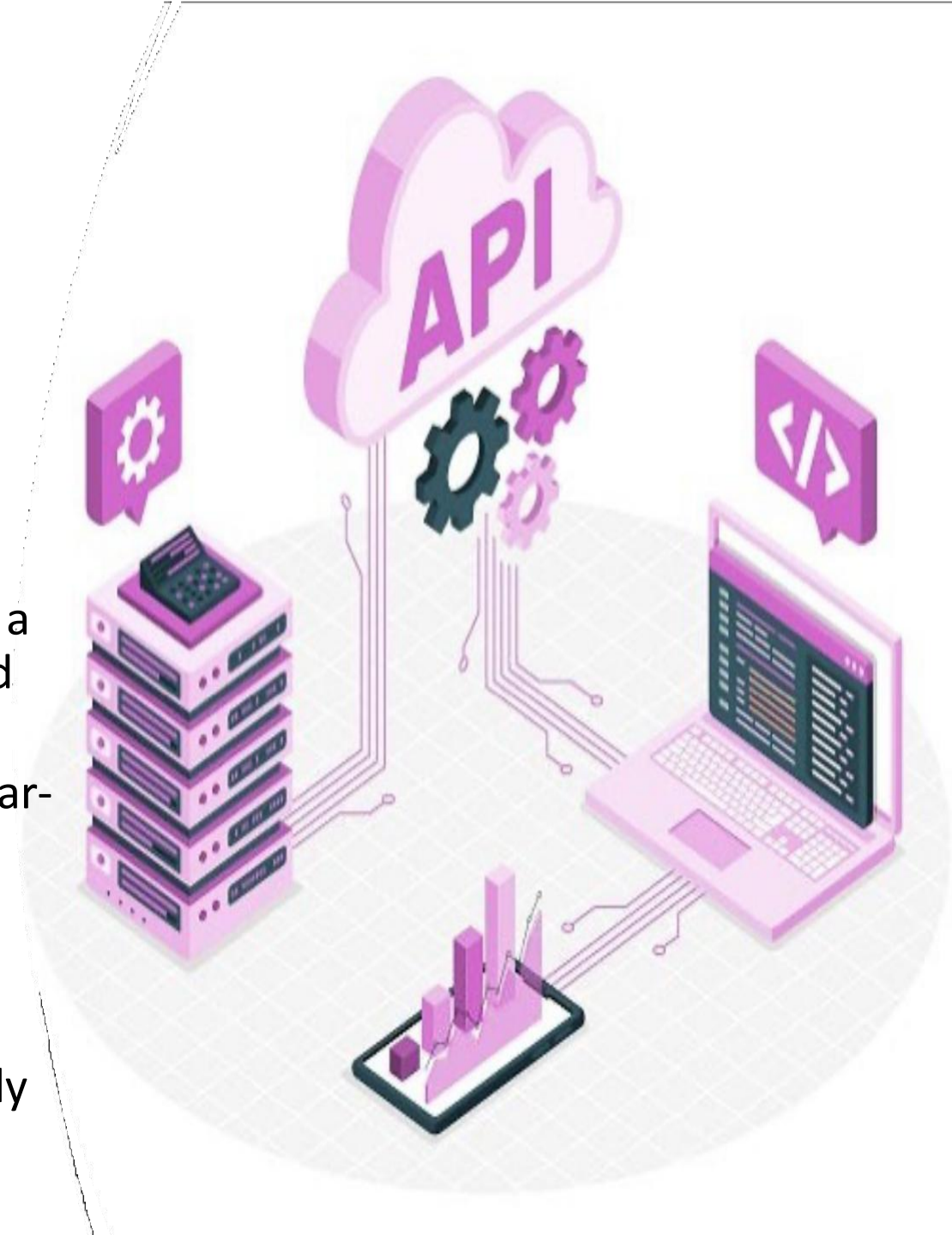
Data Collection through APIs

- What is an API?
 - An API is a set of rules and protocols that allow different software applications to communicate with each other.
 - APIs enable developers to request specific data or perform certain actions from remote servers, usually in a structured format like JSON or XML.



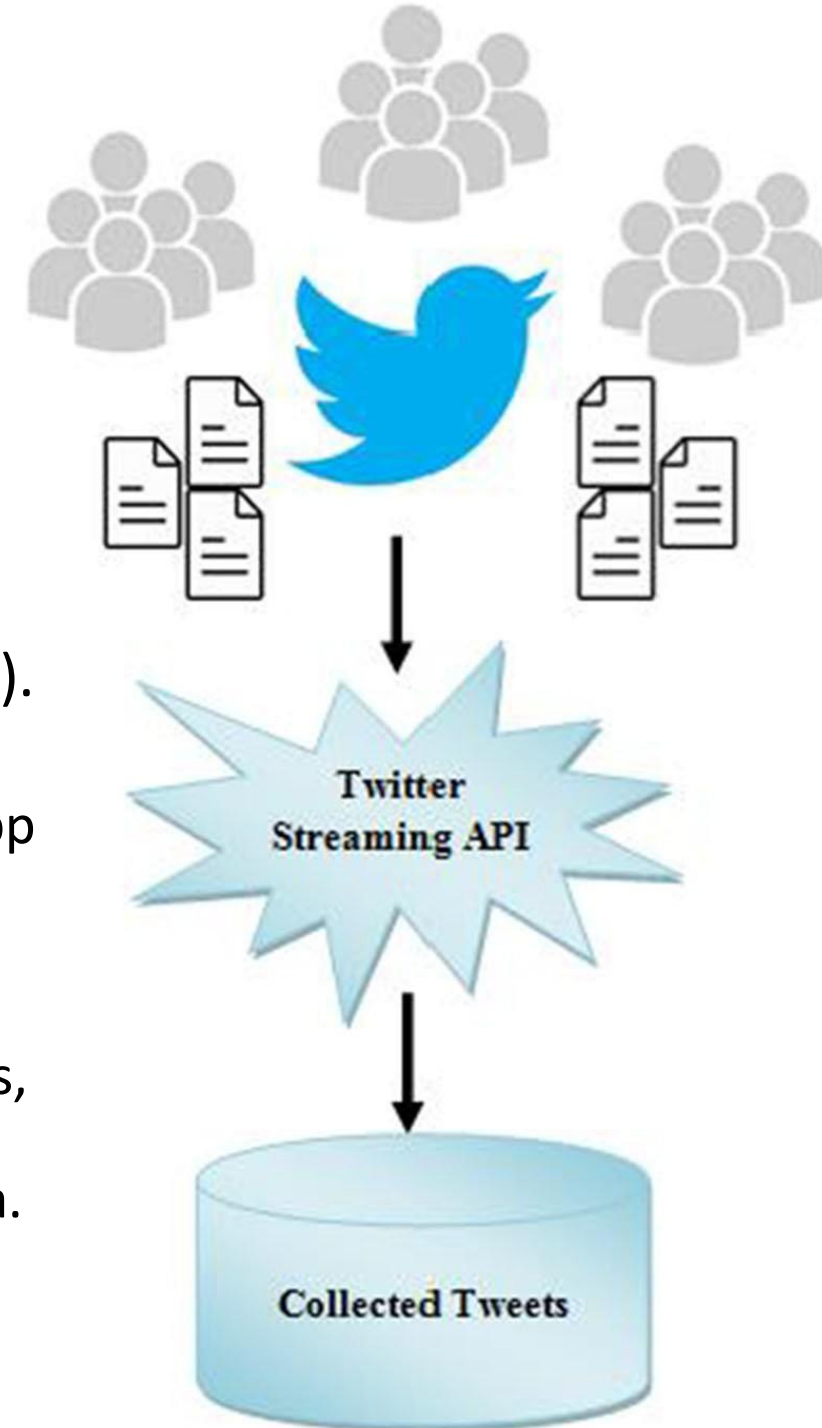
Data Collection through APIs

- Benefits of Using APIs for Data Collection:
 - **Structured Data:** APIs provide structured data in a consistent format, making it easier to extract and use.
 - **Real-Time Data:** Many APIs offer real-time or near-real-time access to the latest information.
 - **Efficiency:** APIs allow you to access specific data points without the need to scrape entire web pages.
 - **Reliability:** Data obtained through APIs is typically more reliable and accurate than web scraping.



Data Collection through APIs

- Example Scenario: Collecting **Tweets** Using the **Twitter API**:
 - API: Twitter API (requires an API key and authentication).
 - Steps:
 - Sign up for a Twitter Developer account and create an app to obtain API keys.
 - Use Python libraries like ***tweepy*** to send requests to the Twitter API endpoints.
 - Request tweets based on specific keywords, user profiles, or hashtags.
 - Parse the JSON response and extract relevant tweet data.
 - Store the data in a structured format for analysis.

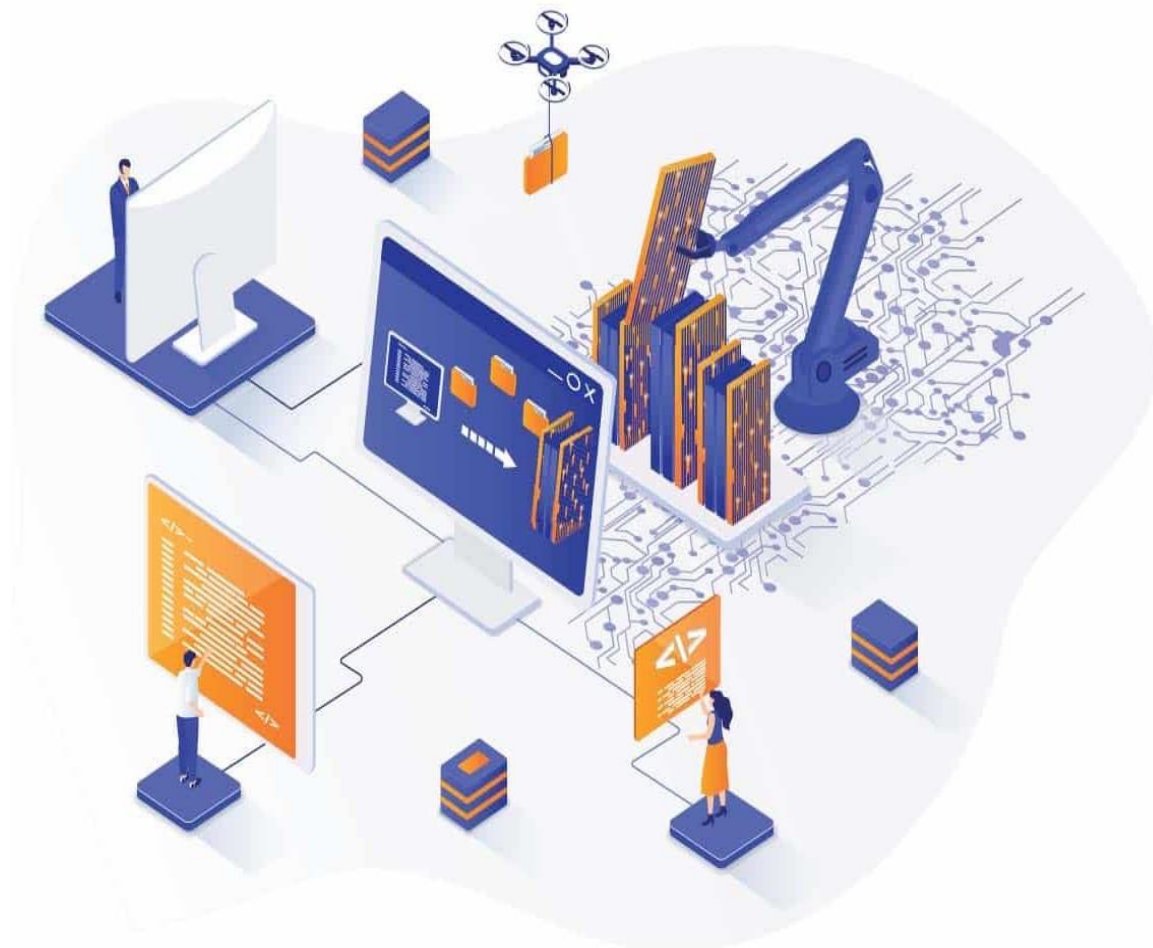


API: Best Practices

- **Rate Limiting:** Respect the API's rate limits to avoid being blocked.
- **Efficient Data Handling:** Parse and store data efficiently to manage large volumes.
- **Authentication:** Securely manage API keys and tokens.
- **Documentation:** Keep clear documentation of your API interactions for future reference.
- **Ethical Considerations:** Ensure compliance with data usage policies and respect user privacy.

API: Challenges

- **Rate Limits:** APIs often limit the number of requests that can be made in a given time period.
- **Data Quality:** Ensuring the accuracy and relevance of the data collected.
- **API Changes:** APIs may change over time, requiring updates to your data collection scripts.
- **Access Restrictions:** Some APIs have restricted access or require payment.



Data Collection by Web Scraping

1,097,398,145

Currently, there are around **1.1 billion** websites in the World. **18%** of these websites are active, **82%** are inactive.

193,527,411

websites are active

252,000

new websites are
created **every day**

10,500

new websites are
created **every hour**

175

new websites are
created **every minute**

3

new websites are
created **every second**

2,000+

new websites by the time
you are done reading this
article

Source: <https://siteefy.com/how-many-websites-are-there/>

Data Collection by Web Scraping

- Many valuable sources of text data are available online
 - news articles, social media content, discussion forums, etc.
- Web scraping enables us to gather relevant and up-to-date data for analysis and model training



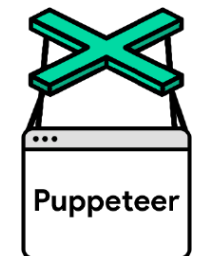
Data Collection by Web Scraping

- Web Scraping Process:
 - HTTP Requests:
 - Web scraping starts with sending HTTP requests to a website's URL.
 - The website responds with HTML content, which contains the data we want to extract.
 - Parsing HTML:
 - The HTML content is parsed to extract specific elements (e.g., paragraphs, headings) containing the desired text data.
 - Data Extraction:
 - Use CSS selectors or XPath expressions to locate and extract the relevant data from the parsed HTML.
 - Data Transformation:
 - The extracted data might require further preprocessing, such as removing HTML tags, handling special characters, and tokenization.
 - Storing Data:
 - Store the extracted and preprocessed data in a structured format (e.g., CSV, JSON) for further analysis or model training.

Data Collection by Web Scraping

- Tools for Web Scraping

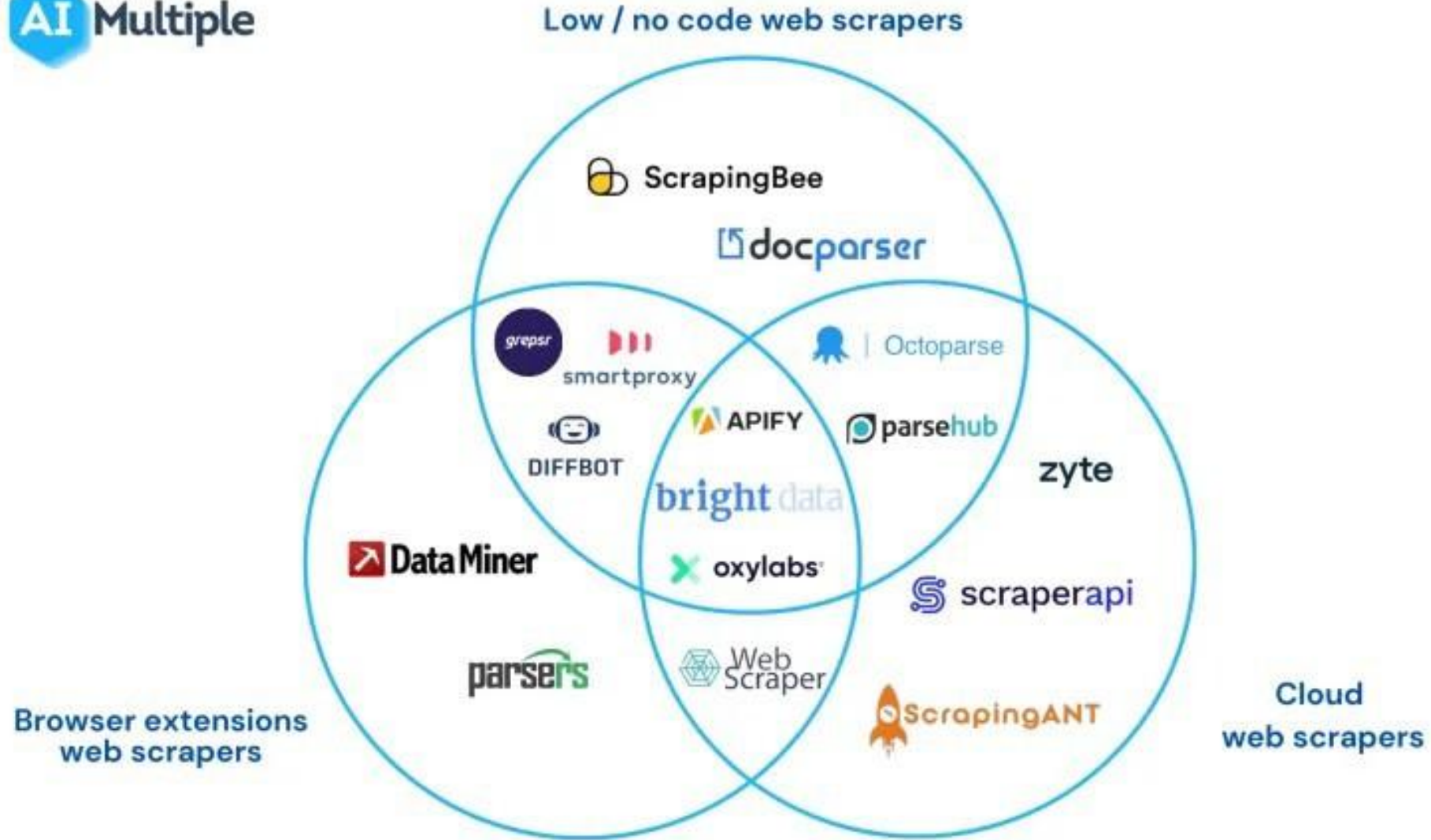
- **Beautiful Soup:** A Python library for parsing HTML and XML documents. It provides easy methods to navigate and search the parsed content.
- **Scrapy:** A more advanced Python framework for web scraping. It offers a structured approach to building web scrapers and handling complex websites.
- **Requests:** A Python library for making HTTP requests, often used in conjunction with BeautifulSoup for simple scraping tasks.
- **Selenium:** A Python library and tool used for automating web browsers to do a number of tasks including web-scraping to extract useful data and information.
- **Puppeteer:** A Node.js library that provides a high-level API to control Chrome or Chromium.



Data Collection by Web Scraping

- Tools for Web Scraping

	Beautiful Soup	Selenium	Scrapy
Performance	Slow for a few tasks	Faster than Beautiful Soup but has its limitations	Pretty fast and has the best performance out of the three
Extensibility	Best suited for small, low-complexity projects & beginners	Best suited for core JavaScript featured website	Best suited for large, complex project. Makes project robust & flexible
Ecosystem	Has a lot of dependencies in the ecosystem	Good ecosystem but can't use proxies	Can use proxies and VPMs and hence suitable for complex projects



Data Collection by Web Scraping

- Example Scenario: News Article Scraping
 - **Objective:** Gather recent news articles for a sentiment analysis project.
 - **Steps:**
 - Identify relevant news websites (e.g., major news outlets, specialized news blogs).
 - Send HTTP requests to article pages, retrieve HTML content.
 - Use BeautifulSoup to parse and extract article titles and content.
 - Preprocess extracted text (remove HTML tags, tokenize, etc.).
 - Store the preprocessed data for analysis.



Scraping: Best Practices

- **Respect robots.txt:** Check the website's robots.txt file to understand which parts of the site can be scrapped.
- **Rate Limiting:** Avoid overloading the server by limiting the rate of requests.
- **Use Proxies:** Distribute requests across multiple IP addresses to avoid getting blocked.
- **Error Handling:** Implement error handling to manage failed requests and retries.
- **Data Cleaning:** Ensure the scraped data is cleaned and structured appropriately.
- **Legal Compliance:** Ensure scraping is compliant with legal guidelines and the website's terms of service.
- **Privacy:** Be cautious with personal data to avoid violating privacy laws.

Scraping: Challenges

- **IP Blocking:** Websites may block IP addresses that send too many requests.
- **Captcha and Security Measures:** Some sites use Captchas and other techniques to prevent automated scraping.
- **Dynamic Content Loading:** Handling content that loads dynamically via JavaScript.
- ...



Data Annotation and Labeling

Data Annotation and Labeling

- Involve adding metadata or tags to raw text data, making it suitable for training supervised NLP models.
- High-quality annotations are crucial for building accurate and effective NLP systems

Importance of Annotation:

- Annotated data serves as ground truth for training and evaluating machine learning models.
- Properly labeled data contributes to the success of tasks like sentiment analysis, text classification, and named entity recognition.



Data Annotation and Labeling

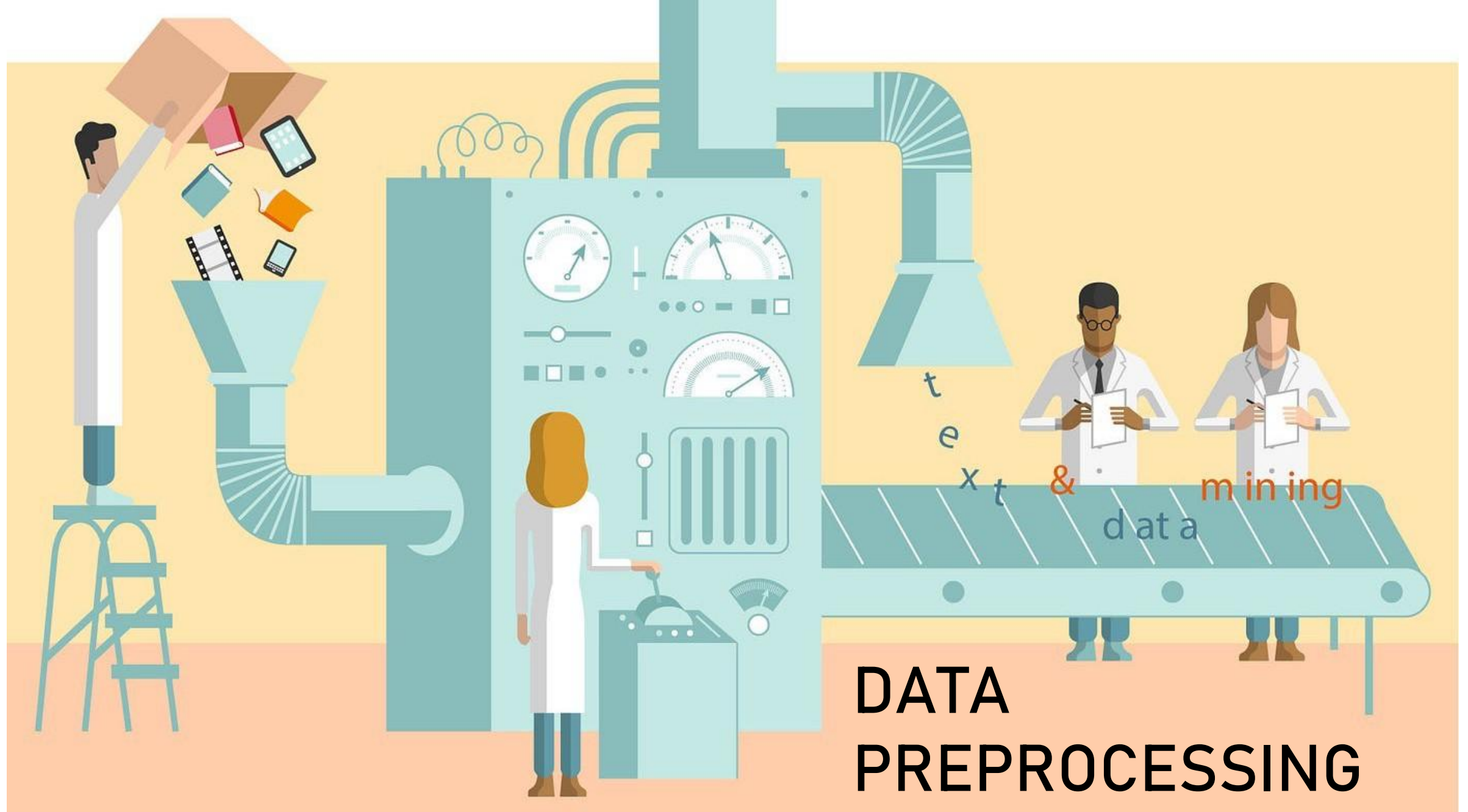
- Annotation Methods:
 - **Manual Annotation:**
 - Human annotators review and label each data point based on predefined guidelines.
 - Ensures high accuracy but can be time-consuming and expensive.
 - **Semi-Automatic Annotation:**
 - Annotators are aided by tools that suggest labels or generate annotations based on certain patterns.
 - Increases efficiency while maintaining human oversight.
 - **Crowdsourcing:**
 - Outsourcing annotation tasks to a large group of workers through platforms like Amazon Mechanical Turk.
 - Cost-effective for large datasets, but quality control is essential.

Data Annotation and Labeling

- Labeling Strategies:
 - **Single-Annotator:** One annotator labels each data point for consistency.
 - **Multiple-Annotator:** Multiple annotators label the same data point to assess inter-annotator agreement.
 - **Majority Voting:** Inconsistent labels are resolved by selecting the most common label among annotators.

Data Annotation and Labeling

- Challenges and Considerations:
 - **Ambiguity:** Some text may have multiple valid interpretations, leading to varying labels.
 - **Subjectivity:** Certain tasks, like sentiment analysis, can be subjective and result in differing annotations.
 - **Bias:** Annotator biases can impact the quality and fairness of labeled data.
 - **Continual Feedback:** Regular communication with annotators helps clarify guidelines and address questions.



Data Preprocessing

- Definition

Process of preparing and cleaning raw text data before it can be analyzed by natural language processing (NLP) algorithms or machine learning models.

- Goal

Transform unstructured text data into a structured and standardized format that can be easily analyzed and understood by computers



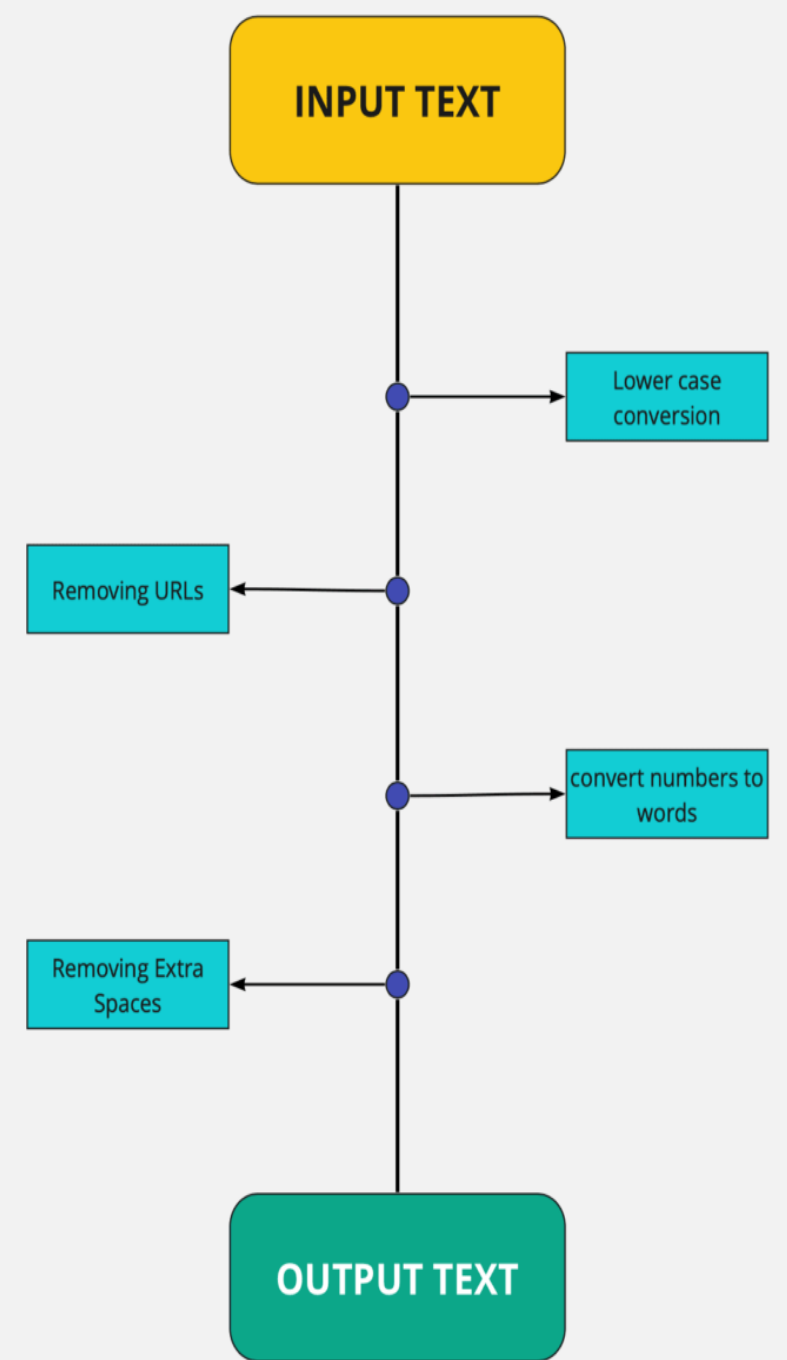


Text Preprocessing

- Why Text Preprocessing is required?
 - Cleaning and standardizing the data
 - Reducing the vocabulary size
 - Enabling efficient text representation
 - Improving model accuracy
 - Facilitating text understanding
 - ...

Text Preprocessing

- Types of text preprocessing techniques
 - Tokenization
 - Lowercasing
 - Stop word removal
 - Stemming and Lemmatization
 - Removing special characters and digits
 - Removing URLs and email addresses
 - Spell checking and correction
 - ...



Tokenization

- Process of breaking up a text into individual words, phrases, symbols or other meaningful elements, called *tokens*
- Tokens are used as the **basic building blocks**
- Involves **identifying the boundaries between words** or other meaningful elements in a text
- A text can be split on whitespace or punctuation marks such as spaces, commas, periods, question marks, and exclamation marks



Tokenization

- For example, consider the following sentence:

"John loves playing soccer with his friends."

- The tokens in this sentence after tokenization might be:
 - John
 - loves
 - Playing
 - soccer
 - with
 - his
 - friends

Lowercasing

- Lowercasing ALL your text data
- One of the simplest and most effective form of text preprocessing
- Applicable to most text mining and NLP problems
- Useful to ensure that the same word is not treated differently because of its capitalization
- Help in cases where your dataset is not very large
- Significantly helps with consistency of expected output

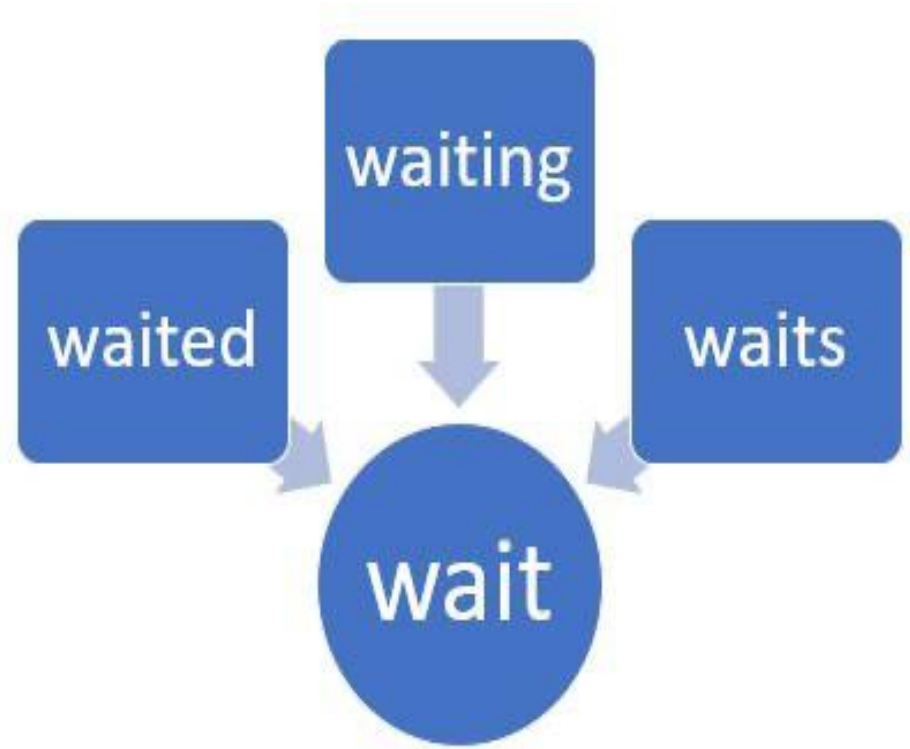


Lowercasing

Raw	Lowercased
Canada CanadA CANADA	canada
TOMCAT Tomcat toMcat	tomcat

Stemming

- Reduce inflected or derived words to their **base form**, known as the **stem**
- Stem may or may not be meaningful
- Purpose is to simplify the text data and reduce the vocabulary size
- Make easier to process and analyze
- Involves applying a set of rules or algorithms to the words in the text data to identify and remove affixes (prefixes and suffixes)
- Porter stemming, Snowball stemming, Lancaster stemming



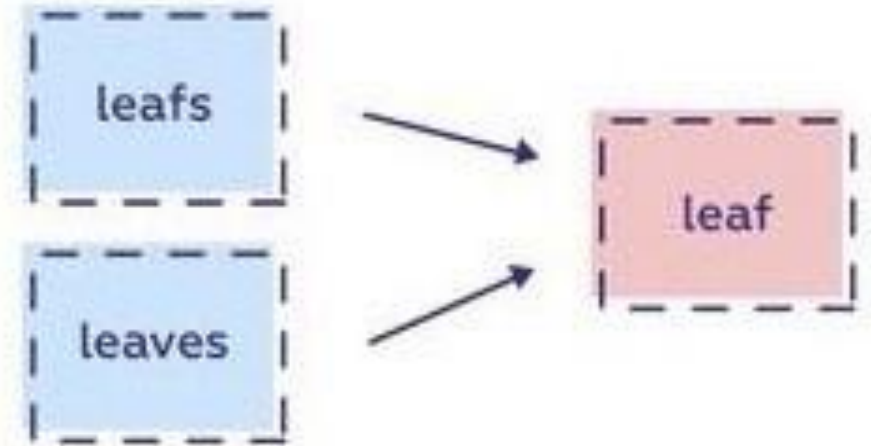
Stemming

	original_word	stemmed_words
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

	original_word	stemmed_word
0	trouble	troubl
1	troubled	troubl
2	troubles	troubl
3	troublesome	troublesom

Lemmatization

- Very similar to stemming
- Reduce inflected or derived words to their base or dictionary form, known as the **lemma**
- Lemma should have a meaning
- Considers the context and part of speech of the word in order to produce the correct lemma
- Involves the use of a lexicon or a set of rules
- More accurate technique than stemming
- More computationally expensive and slower than stemming

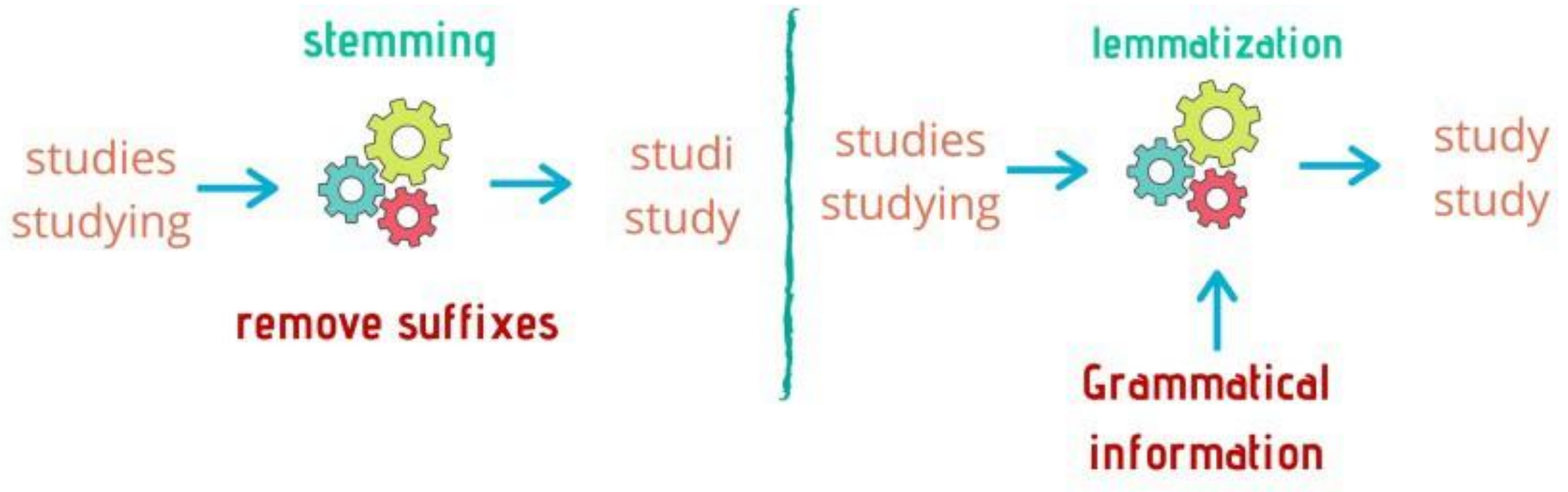


Lemmatization

	original_word	lemmatized_word
0	trouble	trouble
1	troubling	trouble
2	troubled	trouble
3	troubles	trouble

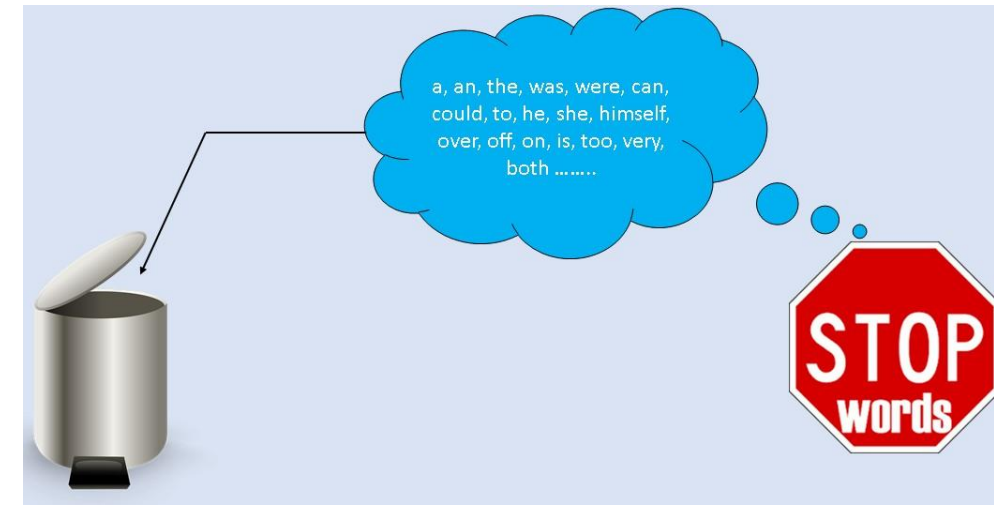
	original_word	lemmatized_word
0	goose	goose
1	geese	goose

Stemming vs Lemmatization



Stop words Removal

- Remove words that are considered irrelevant or redundant in the context of the text data being analyzed
- Stop words are **common words that occur frequently** in language
- "a", "an", "the", "and", "in", "on", and "at", etc. - do not carry much meaning on their own and can be safely ignored in many text analysis tasks
- Purpose is to reduce the size of the vocabulary and simplify the text data
- Can be easily implemented using pre-built lists of stop words or by creating custom lists for specific text analysis tasks



Spell Checking & Correction

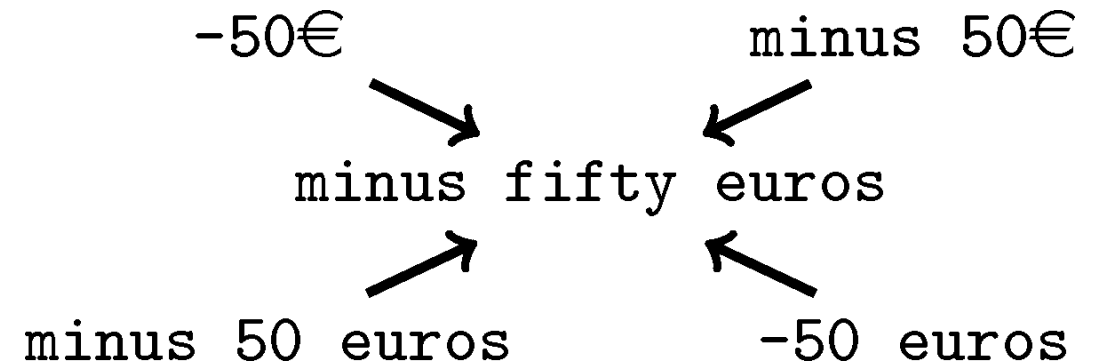
- Identify and correct spelling errors in the text data
- Spelling errors can occur due to typos, misspellings, or other mistakes made during the text entry or transcription process - can affect the accuracy and quality of the text analysis results
- Dictionary-based spell checking
- Rule-based spell checking
- Statistical spell checking
- Machine learning-based spell checking
- Mainly useful in applications that require precise language processing, such as sentiment analysis, named entity recognition, and machine translation

I wasnt sure what to except.

I wasn't sure what to expect.

Text Normalization

- Transform text data into a standardized format that is easier to process and analyze
- Purpose is to reduce the variability in text data and increase its consistency and accuracy
- Case normalization
- Punctuation removal
- Number normalization
- Symbol and abbreviation expansion
- Accent and diacritic removal ...

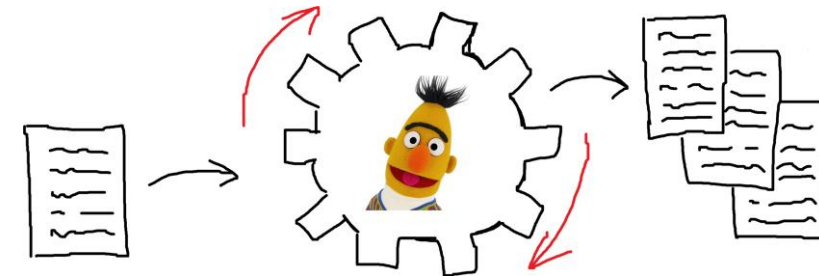


Text Normalization

Raw	Normalized
2moro 2mrrw 2morrow 2mrw tomrw	tomorrow
b4	before
otw	on the way
:) :-) ;-)	smile

Text Enrichment / Augmentation

- Enhance the quality and quantity of text data by adding or generating new content
- Purpose is to increase the diversity and relevance of the text data, and provide additional context and information for text analysis tasks
- **Part-of-speech tagging:** involves labeling each word in the text data with its corresponding part of speech, such as noun, verb, or adjective
- **Named entity recognition:** involves identifying and labeling named entities in the text data, such as people, organizations, and locations
- **Text generation:** involves generating new text data based on the existing text data, using techniques such as language models, recurrent neural networks, and Markov chains



Do you need it all?

- **Must Do:**

- Noise removal
- Lowercasing (can be task dependent in some cases)

- **Should Do:**

- Simple normalization — (e.g. standardize near identical words)

- **Task Dependent:**

- Advanced normalization (e.g. addressing out-of-vocabulary words)
- Stop-word removal
- Stemming / lemmatization
- Text enrichment / augmentation

General Rule of Thumb

- Not all tasks need the same level of preprocessing

Level of Text Preprocessing Needed

	Domain Specific / Noisy Texts	General / Well Written Texts
Lots of data	- <u>Moderate</u> pre-processing - Text enrichment <u>could be helpful</u>	- <u>Light</u> pre-processing - Text enrichment could be helpful, but <u>not critical</u>
Sparse data	- <u>Heavy</u> pre-processing - Text enrichment is <u>important</u>	- <u>Moderate</u> pre-processing - Text enrichment <u>could be helpful</u>